

Plan de Modernizacion

Venta Gamer - Proyecto ARQWEB

Migracion de ASP.NET WebForms (.NET Framework 4.8)
a ASP.NET Core 8 Web API + React 18 + TypeScript

Stack actual	ASP.NET WebForms / .NET Framework 4.8 / SQL Server / ADO.NET
Stack objetivo	.NET 8 LTS / EF Core 8 / Identity+JWT / React 18 + Vite + TS
Plataforma dev	macOS (Apple Silicon o Intel) - cross-platform
Arquitectura	Clean Architecture (4 capas) + monorepo
Base de datos	SQL Server 2022 en Docker
Etapas totales	10 (de 0 a 9), cada una testeable end-to-end

1. Contexto y diagnostico

1.1 Estado actual

El proyecto Venta Gamer es una aplicacion web desarrollada con ASP.NET WebForms sobre .NET Framework 4.8, organizada en 7 proyectos siguiendo arquitectura N-Tier clasica (BE / BLL / DAL / GUI / SECURITY / Services / Interfaces). Implementa patrones Composite (Role/Permission), Observer (multi-idioma) y un sistema propietario de Digitos Verificadores (DVH/DVV) para integridad.

1.2 Inventario detectado

Capa	Componentes
Entidades (BE)	8 clases: Productos, Usuario, RegistroBitacora, DetalleCompra, Idioma, Role, Permission, BackupFileInfo
Logica (BLL)	5 servicios: Bitacora, GestionDb, GestionIdioma, Perfil, Productos
DAL	9 clases con ADO.NET puro + 4 stored procedures
BD	9 tablas + tablas DVH dinamicas
UI WebForms	14 paginas .aspx + MasterPage + 1 Web Service ASMX
Seguridad	LoginManager, SessionManager, CryptoManager (SHA256), DVManager
Roles	Admin, User, WebMaster, Tester (con jerarquia)

1.3 Deuda tecnica y vulnerabilidades

- **Hashing inseguro:** SHA256 sin salt en CryptoManager. Vulnerable a rainbow tables.
- **Sesiones en memoria estatica:** SessionManager no escala, no soporta multi-instancia.
- **Pregunta de seguridad hardcodeada:** 'Cual es su profesor favorito?' = 'Sabato' en PasswordReset.
- **Connection string en codigo fuente:** credenciales en _connection.cs (SA password expuesta).
- **Carrito en Session:** se pierde con el navegador, no auditable.
- **Compras en XML:** Ventas.xml en disco, no transaccional, no consultable.
- **Sin tests:** 0 cobertura automatizada.
- **Solo Windows:** WebForms no corre en Mac/Linux.

2. Stack tecnologico objetivo

2.1 Backend

Tecnologia	Proposito
.NET 8 LTS	Soporte hasta noviembre 2026, ARM nativo en Mac
ASP.NET Core Web API	REST stateless, Swagger/OpenAPI integrado
Entity Framework Core 8	ORM con migrations, code-first, query LINQ
ASP.NET Core Identity	Reemplaza CryptoManager+LoginManager, hashing PBKDF2
JWT Bearer	Reemplaza SessionManager, stateless
FluentValidation	Validacion declarativa de DTOs
Serilog	Logging estructurado (reemplaza Bitacora ad-hoc)
QuestPDF	Generacion de PDF moderna (reemplaza iTextSharp)
Mapster	Mapeo DTO <-> Entity rapido
xUnit + Moq	Testing unitario e integracion

2.2 Frontend

Tecnologia	Proposito
Vite + React 18 + TypeScript	Bundler ultra rapido, type safety
React Router 6	Routing declarativo con loaders
TanStack Query (React Query)	Server state, cache, refetching automatico
Zustand	Client state ligero (auth, UI)
Axios	Cliente HTTP con interceptors para JWT refresh
React Hook Form + Zod	Formularios performantes con validacion typesafe
Tailwind CSS + shadcn/ui	Estilos utility-first + componentes accesibles
i18next	Internacionalizacion (reemplaza Observer de idioma)
Vitest + Testing Library	Tests unitarios
Playwright	Tests end-to-end

2.3 Infraestructura

Tecnologia	Proposito
Docker Compose	Orquestacion local (SQL + backend + frontend)
SQL Server 2022	BD relacional, ARM nativo en Apple Silicon
GitHub Actions (futuro)	CI/CD automatizado

3. Arquitectura objetivo

3.1 Clean Architecture - 4 capas

Adoptamos Clean Architecture para separar responsabilidades y maximizar testabilidad. La regla de dependencias: capas externas conocen las internas, nunca al revés.

Capa	Contiene	Depende de
Domain	Entidades, value objects, enums, interfaces de dominio	Nada
Application	Use cases (CQRS-light), DTOs, validators, interfaces de infra	Domain
Infrastructure	EF Core DbContext, repositorios, servicios externos, Identity	Application + Domain
Api (Web)	Controllers, middleware, configuracion DI, Swagger, JWT	Application + Infrastructure

3.2 Estructura del monorepo

```
modernizacion/  
  docker-compose.yml  
  README.md  
  .editorconfig  
  backend/  
    VentaGamer.sln  
    src/  
      VentaGamer.Api/  
      VentaGamer.Application/  
      VentaGamer.Domain/  
      VentaGamer.Infrastructure/  
    tests/  
      VentaGamer.UnitTests/  
      VentaGamer.IntegrationTests/  
  frontend/  
    package.json  
    vite.config.ts  
    src/  
      api/  
      components/  
      features/  
      routes/  
      lib/
```

4. Plan de etapas

Cada etapa entrega valor probable end-to-end. No avanzamos a la siguiente hasta validar la actual. Tiempo estimado total: 8 a 12 sesiones de trabajo.

Etapa 0: Setup monorepo + Docker SQL Server

Alcance: Crear estructura Clean Architecture, docker-compose con SQL Server 2022, proyecto React vacio.

Criterio de aceptacion: Backend responde en /swagger, React muestra pagina inicial en localhost:5173, BD accesible en localhost:1433.

Etapa 1: EF Core + DbContext + Migration inicial

Alcance: Modelar las 9 entidades con EF Core. Configurar relaciones (Composite roles, idiomas). Migration y seed basico (admin, idiomas, roles, permisos).

Criterio de aceptacion: dotnet ef database update crea las tablas. Endpoint /health retorna OK. Datos seed visibles via SQL.

Etapa 2: Auth: Identity + JWT (PBKDF2)

Alcance: Reemplazar SHA256+SessionManager con ASP.NET Core Identity y JWT Bearer. Endpoints register/login/refresh. Politica de password robusta.

Criterio de aceptacion: Login desde Swagger devuelve JWT valido. Endpoint protegido requiere token. Password hash es PBKDF2.

Etapa 3: API Productos + Frontend catalogo publico

Alcance: CRUD productos con paginacion. React Router + TanStack Query. Pantalla catalogo (home), pantalla login.

Criterio de aceptacion: Catalogo paginado visible en React. Login funciona, JWT se guarda y se envia en headers.

Etapa 4: Carrito persistente + checkout + PDF

Alcance: Migrar carrito de Session a tabla. Endpoint checkout transaccional. Generacion PDF con QuestPDF. Pantallas Carrito y DetalleCompra.

Criterio de aceptacion: Usuario agrega items, confirma compra, descarga PDF. Compra queda registrada en BD.

Etapa 5: Roles, Permisos, ABMperfiles

Alcance: Endpoints para gestion jerarquica de roles. Pantalla admin con asignacion de permisos. Authorization policies en backend.

Criterio de aceptacion: Admin crea/edita roles desde React. Permisos se heredan correctamente segun jerarquia.

Etapa 6: Bitacora con interceptor EF Core

Alcance: Auditoria automatica via SaveChangesInterceptor. Endpoint con filtros. Pantalla bitacora con tabla filtrable.

Criterio de aceptacion: Cada operacion CUD genera registro automatico. Admin filtra por usuario/fecha/modulo.

Etapa 7: Multi-idioma (i18next + BD)

Alcance: i18next en frontend, endpoint que sirve traducciones desde tabla Textoldioma. Cambio en runtime.

Criterio de aceptacion: Usuario cambia idioma desde dropdown, todos los textos se actualizan sin recargar.

Etapa 8: Backup/Restore + integridad (HMAC)

Alcance: Endpoints backup/restore via SMO. Reemplazar DVH por HMAC-SHA256 sobre filas criticas. Pantalla admin.

Criterio de aceptacion: Admin crea/restaura backups. Sistema detecta tampering en filas modificadas externamente.

Etapa 9: Hardening + Deploy docker-compose

Alcance: Refresh tokens, rate limiting, CORS estricto, secrets via env vars, healthchecks. Build de produccion.

Criterio de aceptacion: docker-compose up levanta el stack completo. Sistema pasa OWASP basico.

5. Mapeo legacy -> moderno

Componente legacy	Equivalente moderno
BE/*.cs (entidades anemicas)	Domain/Entities con encapsulacion
BLL/*.cs	Application/UseCases (handlers)
DAL/*.cs (ADO.NET)	Infrastructure/Repositories + EF Core
SECURITY/CryptoManager (SHA256)	ASP.NET Core Identity (PBKDF2)
SECURITY/SessionManager (estatico)	JWT Bearer + ClaimsPrincipal
SECURITY/LoginManager	AuthService con UserManager<>
SECURITY/DVManager (DVH/DVV)	HMAC-SHA256 + audit interceptor
BLL_Bitacora	SaveChangesInterceptor + Serilog
IdiomaSubject (Observer)	i18next + endpoint /translations
Role/Permission (IComposite)	Identity Roles + Claims jerarquicos
GUI/*.aspx + code-behind	React components + API REST
MasterPage	Layout component en React
ProductsService.asmx (SOAP)	Controllers REST con OpenAPI
Session[Carrito]	Tabla Cart + CartItem
Ventas.xml	Tabla Compras + DetalleCompra
iTextSharp 5	QuestPDF
Web.config	appsettings.json + env vars

6. Decisiones tomadas

Arquitectura backend: Clean Architecture (4 proyectos)

Mejor separacion de responsabilidades, mas profesional, facilita testing y mantiene Domain libre de frameworks.

BD en Docker: mssql/server:2022-latest

Soporta ARM nativo en Apple Silicon, compatible con BACKUP/RESTORE y todos los stored procedures.

Datos iniciales: BD vacia + seed basico

Mas rapido. Evita arrastrar passwords SHA256 sin salt del .bak legacy. Productos se cargan via API en pruebas.

Autenticacion: JWT stateless + refresh tokens

Estandar industria, escalable, compatible con SPAs y mobile.

Frontend state: TanStack Query + Zustand

Server state separado de client state. Cache automatico, menos boilerplate que Redux.

PDF: QuestPDF

Reemplaza iTextSharp 5 (licencia AGPL pesada). API fluida, soporte .NET 8 nativo, licencia MIT.

7. Riesgos y mitigaciones

Curva de aprendizaje React/TS: Etapas pequenas con pantallas simples. Documentacion incremental.

Migracion de jerarquia de roles (Composite): Mantener estructura ROLES + ROLESJERARQUIA con un servicio que materialice claims al login.

Tests de Identity con BD real: TestContainers para SQL Server en integracion.

Performance de TanStack Query con muchos items: Paginacion server-side + virtualizacion (TanStack Virtual).

Compatibilidad SQL Server en Mac M1/M2/M3: Imagen oficial 2022 ya tiene soporte ARM nativo (verificado).

8. Entregables por etapa

Al cierre de cada etapa se entrega: codigo funcionando, README actualizado con comandos para probar, capturas de pantalla cuando aplique, y un commit limpio. El proyecto legacy permanece intacto en la raiz.